

MATHEMATICAL APPENDICES TO “THE PROBABILITY OF BACKTEST OVERFITTING”

David H. Bailey
Marcos López de Prado

Jonathan M. Borwein
Jim Zhu

Source code available at www.QuantResearch.info
Paper available at <http://ssrn.com/abstract=2326253>

A.1. TEST CASES

We will compare how PBO responds to several test cases: Full, high and low overfit. These cases are created by setting a matrix $\mathbf{M}$ with $N-1$ trials of length T and null Sharpe ratio. If we add to that matrix $\mathbf{M}$ a trial with Sharpe ratio zero, the strategy selection procedure will deliver a full overfit, because selecting the strategy with highest Sharpe ratio IS in this context cannot improve performance OOS over the median (zero). If we add to $\mathbf{M}$ a trial with Sharpe ratio 1, selecting the strategy with the highest Sharpe ratio IS may still improve performance OOS over the median (zero), giving us the high overfit case. If we add to $\mathbf{M}$ a trial with Sharpe ratio 2, selecting the strategy with the highest Sharpe ratio IS will likely improve performance OOS over the median (zero), giving us the low overfit case.

A.1.1. TEST CASE 1: FULL OVERFIT

We create a matrix $\mathbf{M}$ where $T=1000$, $N=100$ as follows.

1. For each trial, $n=1, \dots, 100$:
 - a. Form a vector of T random draws from a standard Normal distribution.
 - b. Re-scale and re-centre the vector so that its Sharpe ratio is 0.
2. Combine the N vectors into a matrix $\mathbf{M}$.

If we choose IS the trial with highest Sharpe ratio, this cannot be expected to outperform OOS. If our procedure is accurate, we should obtain that $\phi \rightarrow 1$, indicating that in virtually all cases the “optimal” strategy IS happened to underperform OOS the median of trials. In effect, our simulations show that CSCV correctly determined that backtests are almost certain to overfit in this situation.

Increasing the sample size to $T=2000$ has no effect, and we still obtain $\phi \rightarrow 1$. Similarly, increasing the number of trials to $N=200$ still produces $\phi \rightarrow 1$.

A.1.2. TEST CASE 2: HIGH OVERFIT

We create a matrix $\mathbf{M}$ where $T=1000$, $N=100$ as follows.

1. For each trial, $n=1, \dots, 100$:
 - a. Form a vector of T random draws from a standard Normal distribution.
 - b. Re-scale and re-centre that vector so that its Sharpe ratio is 0.
2. Re-scale and re-centre the N th vector so that its Sharpe ratio is 1.
3. Combine the N vectors into a matrix $\mathbf{M}$.

Because one of the trials is expected to succeed OOS, PBO is not 1. At the same time, a random sample with a Sharpe ratio of 0 over T observations is likely to produce IS a Sharpe ratio above 1 over $\frac{T}{2}$ observations (see Bailey et al. [2013]). Accordingly, it is very likely that the strategy selection procedure will choose one trial with a high Sharpe ratio IS, only to underperform OOS the median of trials. The conclusion is that PBO will still be high in this scenario. Our Monte Carlo simulations confirm that intuition, by computing overfitting probabilities between 0.7 and 0.8.

Increasing the number of trials to $N=200$ slightly increases ϕ , which now ranges between 0.75 and 0.85. The reason is, as more trials are added, the risk of overfitting increases, thus the importance that the researcher reports all the possibilities truly tested.

Increasing the sample size to $T=2000$ reduces PBO to values between 0.4 and 0.5, because the larger the number of available observations, the more informative is the performance IS. These empirical findings are consistent with Proposition 1 and Theorem 1, discussed in Bailey et al. [2013].

A.1.3. TEST CASE 3: LOW OVERFIT

We create a matrix $\mathbf{M}$ where $T=1000$, $N=100$ as follows.

1. For each trial, $n=1, \dots, 100$:
 - a. Form a vector of T random draws from a standard Normal distribution.
 - b. Re-scale and re-centre that vector so that its Sharpe ratio is 0.
2. Re-scale and re-centre the N th vector so that its Sharpe ratio is 2.
3. Combine the N vectors into a matrix $\mathbf{M}$.

Given that one of the trials performs significantly better than the rest over T observations, we expect a greater probability of it being selected IS over $\frac{T}{2}$ observations. Still, there is a non-null probability that the strategy selection procedure fails to do so, because it is still possible for a subsample of that trial to underperform IS the subsample of one of the other trials, hence leading to overfitting. Accordingly, our simulations estimate overfitting probabilities ranging between 0.1 and 0.2.

Increasing the number of trials to $N=200$ slightly increases ϕ , which now ranges between 0.2 and 0.3. Increasing the sample size to $T=2000$ reduces PBO to values between 0 and 0.04, just as we argued in Bailey et al. [2013].

The reader may draw a parallel between these results and her knowledge of overfitting in regression models. As it is documented in most statistics textbooks, the probability of overfitting a regression model increases as the number of degrees of freedom decreases. Our $\mathbf{M}$ matrix has its regression analogue in the $\mathbf{X}$ matrix of explanatory (or exogenous) variables, shaped in T rows (time observations) and N factors (columns), used to fit some other explained variable. In such regression model, the number of degrees of freedom is $T-N$. The larger T is, the more degrees of freedom, and the lower the risk of overfitting. Conversely, the larger N is, the less degrees of freedom and the higher the risk of overfitting. In conclusion, ϕ behaves as we would have expected in the familiar case of regression models.

A.2. ACCURACY OF THE TEST

In the previous Section, as we increased the Sharpe ratio of the N th trial above the other trials in matrix $\mathbf{M}$, we observed a decrease in PBO. Decreasing N and increasing T had a similar effect. Thus, overfitting estimates in the test cases above seem reasonable in an ordinal sense. The question remains, does the actual estimate correspond to the probability of the selected strategy to underperform the median of trials OOS? We evaluate the accuracy of our CSCV procedure to determine PBO in two different ways: Via Monte Carlo (MC) simulations, and applying Extreme Value Theory (EVT).

A.2.1. ACCURACY BY MC

In order to determine the MC accuracy of our PBO estimate, we have generated 1,000 matrices $\mathbf{M}$ (experiments) for various test cases of order $(T \times N) = (1000 \times 100)$, and computed the proportion of experiments that yielded an OOS performance below the median. If our CSCV procedure is accurate, the PBO that we estimated by resampling slices of a single matrix $\mathbf{M}$ should be close to the probability derived from generating 1,000 matrices $\mathbf{M}$ and computing the proportion of IS optima that underperformed the median OOS.

Snippet 1 lists a code written in Python that estimates PBO via Monte Carlo.

```
#!/usr/bin/env python
# On 20130704 by lopezdeprado@lbl.gov
#-----
def testAccuracy_MC(sr_base,sr_case):
    # Test the accuracy of CSCV against hold-out
    # It generates numTrials random samples and directly computes the ...
    # ... proportion where OOS performance was below the median.
    length,numTrials,numMC=1000,100,1000
    pathOutput='H:/Studies/Quant #23/paper/'
    #1) Determine mu,sigma
    mu_base,sigma_base=sr_base/(365.25*5/7.),1/(365.25*5/7.)**.5
    mu_case,sigma_case=sr_case/(365.25*5/7.),1/(365.25*5/7.)**.5
    hist,probOverfit=[],0
    #2) Generate trials
    for m in range(numMC):
        for i in range(1,numTrials):
            j=np.array([gauss(0,1) for j in range(length)])
            j*=sigma_base/np.std(j) # re-scale
            j+=mu_base-np.mean(j) # re-center
            j=np.reshape(j,(j.shape[0],1))
            if i==1:pnl=np.copy(j)
            else:pnl=np.append(pnl,j,axis=1)
    #3) Add test case
    j=np.array([gauss(0,1) for j in range(length)])
    j*=sigma_case/np.std(j) # re-scale
```

```

j+=mu_case-np.mean(j) # re-center
j=np.reshape(j,(j.shape[0],1))
pnl=np.append(pnl,j,axis=1)
#4) Run test
# Reference distribution
mu_is=[np.average(pnl[:length/2,i]) for i in range(pnl.shape[1])]
sigma_is=[np.std(pnl[:length/2,i]) for i in range(pnl.shape[1])]
mu_oos=[np.average(pnl[length/2:,i]) for i in range(pnl.shape[1])]
sigma_oos=[np.std(pnl[length/2:,i]) for i in range(pnl.shape[1])]
sr_is=[mu_is[i]/sigma_is[i] for i in range(len(mu_is))]
sr_oos=[mu_oos[i]/sigma_oos[i] for i in range(len(mu_oos))]
print m,sr_is.index(max(sr_is)),max(sr_is), \
      sr_oos[sr_is.index(max(sr_is))]
sr_oos_=sr_oos[sr_is.index(max(sr_is))]
hist.append(sr_oos_)
if sr_oos_<np.median(sr_oos):probOverfit+=1
probOverfit/=float(numMC)
print probOverfit
return

```

Snippet 1 – Python code for estimating PBO via Monte Carlo

A.2.2. ACCURACY BY EVT

Backtesting a number N of alternative strategy configurations and selecting the trial that exhibits maximum performance IS sets the background for applying Extreme Value Theory. From Bailey et al. [2014] we know that Sharpe ratio estimates asymptotically follow a Gaussian distribution. Proposition 1 discussed the distribution of the maximum of N independent random variables. As part of its proof, we applied the Fisher-Tippett-Gnedenko theorem to the Gaussian distribution, and concluded that the maximum performance IS among N alternative backtests can be approximated through a Gumbel distribution.

More formally, suppose a set of backtests where $N=100$, $T=1000$, $SR_n = 0$ for $n=1, \dots, N-1$ and $SR_N = \widetilde{SR} > 0$. The sample length is divided in two sets of equal size $(\frac{T}{2})$, IS and OOS. A strategy with $SR_n = 0$ is selected when its IS SR is greater than the IS SR of the strategy with $SR_N = \widetilde{SR}$. Because the Sharpe ratio has been globally constrained for the whole sample by re-scaling and re-centering, and IS has the same length as OOS, $SR_{OOS}^* \approx SR - SR_{IS}^*$. By virtue of this global constraint, the following propositions can be used to estimate PBO:

- i. The median of all Sharpe ratios OOS is null, $Me[SR_{OOS}] = 0$.
- ii. Selecting a strategy with $SR_n = 0$ leads to $SR_{OOS}^* \approx -\max_N < Me[SR_{OOS}]$ iif $SR_{IS}^* > 0$.
- iii. Selecting a strategy with $SR_N = \widetilde{SR}$ leads to $SR_{OOS}^* \approx \widetilde{SR} - SR_{IS}^*$, where $E[SR_{OOS}^*] > Me[SR_{OOS}]$ iif $SR_{IS}^* \in (-\infty, 2\widetilde{SR})$ and $E[SR_{OOS}^*] \leq Me[SR_{OOS}]$ iif $SR_{IS}^* \in [2\widetilde{SR}, \infty)$.
- iv. Computing SR_{IS} fully determines SR_{OOS} as a result of the global constraint. Thus, $V[SR_{IS}] = V[SR] = \frac{1+\frac{1}{2}SR^2}{T}$, and $V[SR_{OOS}] = 0$.

Our goal is to compute the probability that the strategy with maximum Sharpe ratio IS performs below the median of OOS Sharpe ratios. First, we need to calibrate the parameters of the Gumbel distribution associated with a set of Gaussian random variables. Suppose a random variable $max_N = \max(\{SR_n | n = 1, \dots, N-1\})$, where SR_n is the Sharpe ratio estimated through a backtest for trial n . We know that the Gaussian distribution belongs to the Maximum Domain of Attraction of the Gumbel distribution, thus $max_N \sim \Lambda[\alpha, \beta]$, where α, β are the normalizing constants and Λ is the CDF of the Gumbel distribution. Next, we derive the values of these normalizing constants. It is known that the mean and standard deviation of a Gumbel distribution are

$$\begin{aligned} E[max_N] &= \alpha + \gamma\beta \\ \sigma[max_N] &= \frac{\beta\pi}{\sqrt{6}} \end{aligned} \quad (1)$$

where γ is the Euler-Mascheroni constant. Applying the method of moments, we can derive the following:

- Given an estimate of $\hat{\sigma}[max_N]$, we can estimate $\hat{\beta} = \frac{\hat{\sigma}[max_N]\sqrt{6}}{\pi}$.
- Given an estimate of $\hat{E}[max_N]$, and the previously obtained $\hat{\beta}$, we can estimate $\hat{\alpha} = \hat{E}[max_N] - \gamma\hat{\beta}$.

These parameters allow us to model the distribution of the maximum Sharpe ratio IS out of a set of $N-1$ trials. PBO can then be directly computed as $\phi = \phi_1 + \phi_2$, where:

$$\begin{aligned} \phi_1 &= \int_{-\infty}^{2\widetilde{SR}} N \left[SR, \widetilde{SR}, \frac{1 + \frac{1}{2}\widetilde{SR}^2}{T} \right] (1 - \Lambda[\max(0, SR), \alpha, \beta]) dSR \\ \phi_2 &= \int_{2\widetilde{SR}}^{\infty} N \left[SR, \widetilde{SR}, \frac{1 + \frac{1}{2}\widetilde{SR}^2}{T} \right] dSR \end{aligned} \quad (2)$$

Probability ϕ_1 accounts for selecting IS a strategy with $SR_n = 0$, as a result of $SR_{N,IS} < SR_{IS}^*$. As we argued earlier, in this situation $SR_{OOS}^* \approx -max_N < Me[SR_{OOS}] = 0$ iff $SR_{IS}^* > 0$, hence the $\max(0, SR)$ used to evaluate the Gumbel distribution. The integral has an upper boundary in $2\widetilde{SR}$ because beyond that point all trials lead to $SR_{OOS}^* < Me[SR_{OOS}]$, including the N th trial. That probability is accounted for by ϕ_2 , which has a lower boundary of integration in $2\widetilde{SR}$. Snippet 2 implements the numerical integration of Eq. (16). As we will see in Section A.2.3, the EVT estimates of PBO derived from Eq. (16) are in agreement with the MC estimates.

```
#!/usr/bin/env python
# On 20130704 by lopezdeprado@lbl.gov
#-----
def testAccuracy_EVT(sr_base,sr_case):
    # Test accuracy by numerical integration
    # It does the same as testAccuracy_MC, but through numerical integration ...
```

```

# ... of the base and case distributions.
#1) Parameters
parts,length,freq,minX,trials=1e4,1000,365.25*5/7.,-10,100
emc=0.57721566490153286 # Euler-Mascheroni constant
#2) SR distributions
dist_base=[sr_base,((freq+.5*sr_base**2)/length)**.5]
dist_case=(sr_case,((freq+.5*sr_case**2)/length)**.5)
#3) Fit Gumbel (method of moments)
maxList=[]
for x in range(int(parts)):
    max_=max([gauss(dist_base[0],dist_base[1]) for i in range(trials)])
    maxList.append(max_)
dist_base[1]=np.std(maxList)*6**.5/math.pi
dist_base[0]=np.mean(maxList)-emc*dist_base[1]
#4) Integration
prob1=0
for x in np.linspace(minX*dist_case[1],2*dist_case[0]-sr_base,parts):
    f_x=ss.norm.pdf(x,dist_case[0],dist_case[1])
    F_y=1-ss.gumbel_r.cdf(x,dist_base[0],dist_base[1])
    prob1+=f_x*F_y
prob1*=(2*dist_case[0]-sr_base-minX*dist_case[1])/parts
prob2=1-ss.norm.cdf(2*dist_case[0]-sr_base,dist_case[0],dist_case[1])
print dist_base,dist_case
print prob1,prob2,prob1+prob2
return

```

Snippet 2 – Python code for computing PBO by EVT

A.2.3. EMPIRICAL STUDY OF ACCURACY

We are finally ready to evaluate the accuracy of CSCV's PBO. To achieve that, we will compare CSCV's PBO estimates against the two alternative benchmarks described in Sections A.2.1 (MC) and A.2.2 (EVT). Table 1 reports the results for a wide range of combinations of $\widehat{SR}$ (SR_Case), T and N . Without loss of generality, $SR_n = 0$ for $n=1, \dots, N-1$. We do not need to consider alternative values of SR_n , because the likelihood of selecting the wrong strategy is a function of $\widehat{SR} - SR_n$, not the absolute level of SR_n .

Monte Carlo results were computed on 1,000 experiments. The proportion of IS optimal selections that underperformed OOS is reported in Prob_MC. Column Prob_EVT reports the corresponding PBO estimates, derived from Eq. (16). Because these latter results are derived from the actual distribution of the maximum SR, they are more accurate than the Monte Carlo estimates. In any case, EVT and MC results are very close, with a maximum absolute deviation of 4.2%.

We have computed CSCV's PBO on 1,000 randomly generated matrices $\mathbf{M}$ for every parameter combination $(\widehat{SR}, T, N)$. Therefore, we have obtained 1,000 independent estimates of PBO for every parameter combination, with a mean and standard deviation reported in columns

Mean_CSCV and Std_CSCV. This is not to be mistaken with the Monte Carlo result, which produced a single estimate of PBO out of 1,000 randomly generated matrices $\mathbf{M}$.

A comparison of the Mean_CSCV probability with the EVT result gives us an average absolute error of 2.1%, with a standard deviation of 2.9%. The maximum absolute error is 9.9%. That occurred for the combination $(\widetilde{SR}, T, N) = (3, 500, 500)$, whereby CSCV gave a more conservative estimate (24.7% instead of 14.8%). There is only one case where CSCV underestimated PBO, with an absolute error of 0.1%. The median error is only 0.7%, with a 5%-tile of 0% and a 95%-tile of 8.51%.

In conclusion, CSCV provides accurate estimates of PBO, with relatively small errors on the conservative side.

SR_Case	T	N	Mean_CSCV	Std_CSCV	Prob_MC	Prob_EVT	CSCV-EVT
0	500	500	1.000	0.000	1.000	1.000	0.000
0	1000	500	1.000	0.000	1.000	1.000	0.000
0	2500	500	1.000	0.000	1.000	1.000	0.000
0	500	100	1.000	0.000	1.000	1.000	0.000
0	1000	100	1.000	0.000	1.000	1.000	0.000
0	2500	100	1.000	0.000	1.000	1.000	0.000
0	500	50	1.000	0.000	1.000	1.000	0.000
0	1000	50	1.000	0.000	1.000	1.000	0.000
0	2500	50	1.000	0.000	1.000	1.000	0.000
0	500	10	1.000	0.001	1.000	1.000	0.000
0	1000	10	1.000	0.000	1.000	1.000	0.000
0	2500	10	1.000	0.000	1.000	1.000	0.000
1	500	500	0.993	0.007	0.991	0.994	-0.001
1	1000	500	0.893	0.032	0.872	0.870	0.023
1	2500	500	0.561	0.022	0.487	0.476	0.086
1	500	100	0.929	0.023	0.924	0.926	0.003
1	1000	100	0.755	0.034	0.743	0.713	0.042
1	2500	100	0.371	0.034	0.296	0.288	0.083
1	500	50	0.870	0.031	0.878	0.859	0.011
1	1000	50	0.666	0.035	0.628	0.626	0.041
1	2500	50	0.288	0.047	0.199	0.220	0.068
1	500	10	0.618	0.054	0.650	0.608	0.009
1	1000	10	0.399	0.054	0.354	0.360	0.039
1	2500	10	0.123	0.048	0.093	0.086	0.036
2	500	500	0.679	0.037	0.614	0.601	0.079
2	1000	500	0.301	0.038	0.213	0.204	0.097
2	2500	500	0.011	0.011	0.000	0.002	0.009
2	500	100	0.488	0.035	0.413	0.405	0.084
2	1000	100	0.163	0.045	0.098	0.099	0.065
2	2500	100	0.004	0.006	0.002	0.001	0.003
2	500	50	0.393	0.040	0.300	0.312	0.081
2	1000	50	0.113	0.044	0.068	0.066	0.047
2	2500	50	0.002	0.004	0.000	0.000	0.002
2	500	10	0.186	0.054	0.146	0.137	0.049
2	1000	10	0.041	0.027	0.011	0.023	0.018
2	2500	10	0.000	0.001	0.000	0.000	0.000
3	500	500	0.247	0.043	0.174	0.148	0.099
3	1000	500	0.020	0.017	0.005	0.005	0.015
3	2500	500	0.000	0.000	0.000	0.000	0.000
3	500	100	0.124	0.042	0.075	0.068	0.056
3	1000	100	0.007	0.008	0.001	0.002	0.005
3	2500	100	0.000	0.000	0.000	0.000	0.000
3	500	50	0.088	0.037	0.048	0.045	0.043
3	1000	50	0.004	0.006	0.002	0.001	0.003
3	2500	50	0.000	0.000	0.000	0.000	0.000
3	500	10	0.028	0.022	0.010	0.015	0.013
3	1000	10	0.001	0.002	0.000	0.001	0.000
3	2500	10	0.000	0.000	0.000	0.000	0.000

Table 1 – CSCV's accuracy